UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

Asynchronous Communication and Real-Time Notification

Distributed Systems

Author: Charyyeva Altyn
Group: 30242

Faculty of Automation and Computer Science

5 December 2024

Contents

1 Asynchronous communication using queues **2**

2 RabbitMQ: A Messaging Queue Platform **3**

3 Conceptual architecture of the distributed system **3**

4 UML Deployment diagram **4**

1 Asynchronous communication using queues

Asynchronous communication is a method where the sender and receiver of a message do not need to interact with each other in real-time. This approach allows for decoupling systems, meaning they can operate independently without waiting for responses immediately, which is especially useful in distributed systems where different components might operate at different speeds or may be temporarily unavailable.

Queues play a vital role in this kind of communication. Here's how it works:

1. **Message Producer (Sender):** The sender creates and sends a message to a queue. The sender does not need to wait for the message to be processed by the receiver; it simply places the message in the queue and can move on to other tasks.
2. **Message Queue (Broker):** The queue acts as an intermediary that temporarily holds messages. It ensures that messages are stored reliably until the receiver is ready to process them. Queues provide fault tolerance, so if the receiver goes offline, the message remains in the queue until it can be consumed.
3. **Message Consumer (Receiver):** The receiver retrieves messages from the queue when it is ready to process them. This can happen immediately or after some time, depending on the consumer's availability or workload.

Benefits of Asynchronous Communication Using Queues

- **Decoupling:** The producer and consumer are loosely coupled. The producer sends a message to the queue without needing to know when or how the consumer will process it.
- **Load Balancing:** Multiple consumers can process messages from the same queue, enabling load distribution. This helps with scaling systems.
- **Resilience and Fault Tolerance:** Queues store messages reliably, even if one component of the system fails. The system can recover and continue processing without data loss.
- **Improved Efficiency:** Since the sender doesn't wait for the receiver's response, it can perform other tasks, leading to better resource utilization and parallelism.

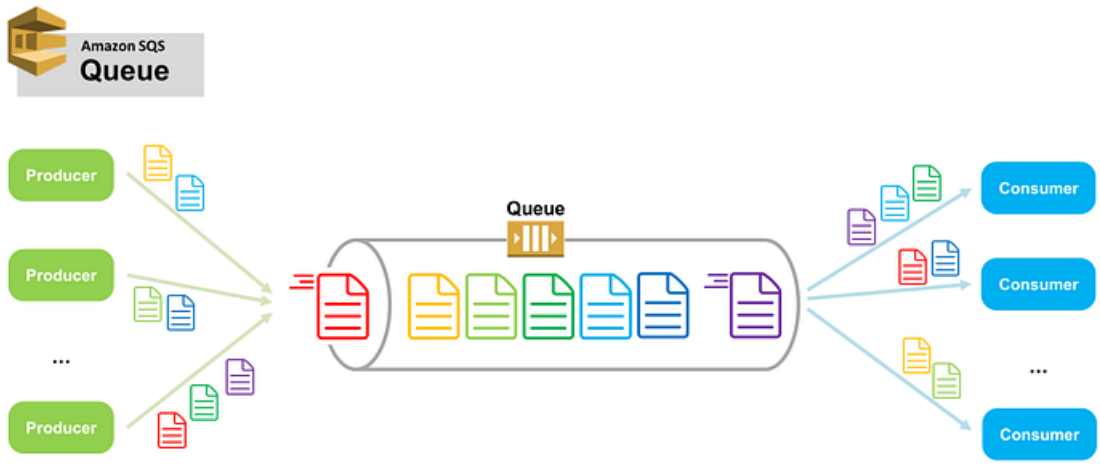

Figure 1: Queue: Producer-Consumer

2 RabbitMQ: A Messaging Queue Platform

RabbitMQ is one of the most popular open-source message brokers, specifically designed for asynchronous communication. It implements the Advanced Message Queuing Protocol (AMQP), though it also supports other protocols like STOMP, MQTT, and more.

3 Conceptual architecture of the distributed system

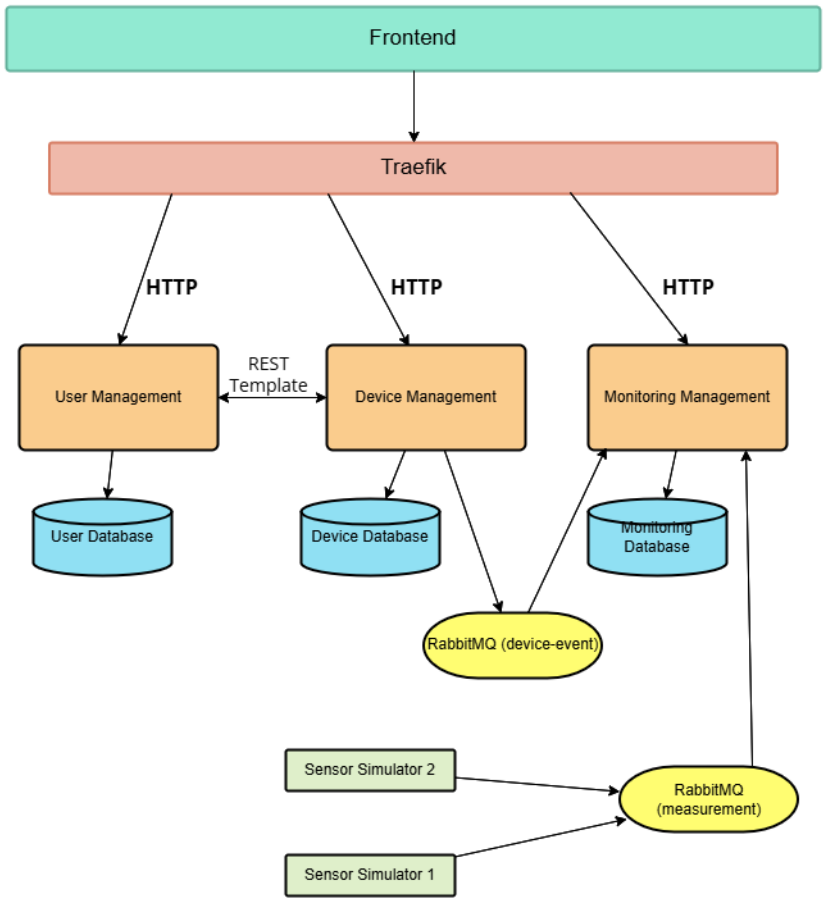

Figure 2: Conceptual Diagram

4 UML Deployment diagram

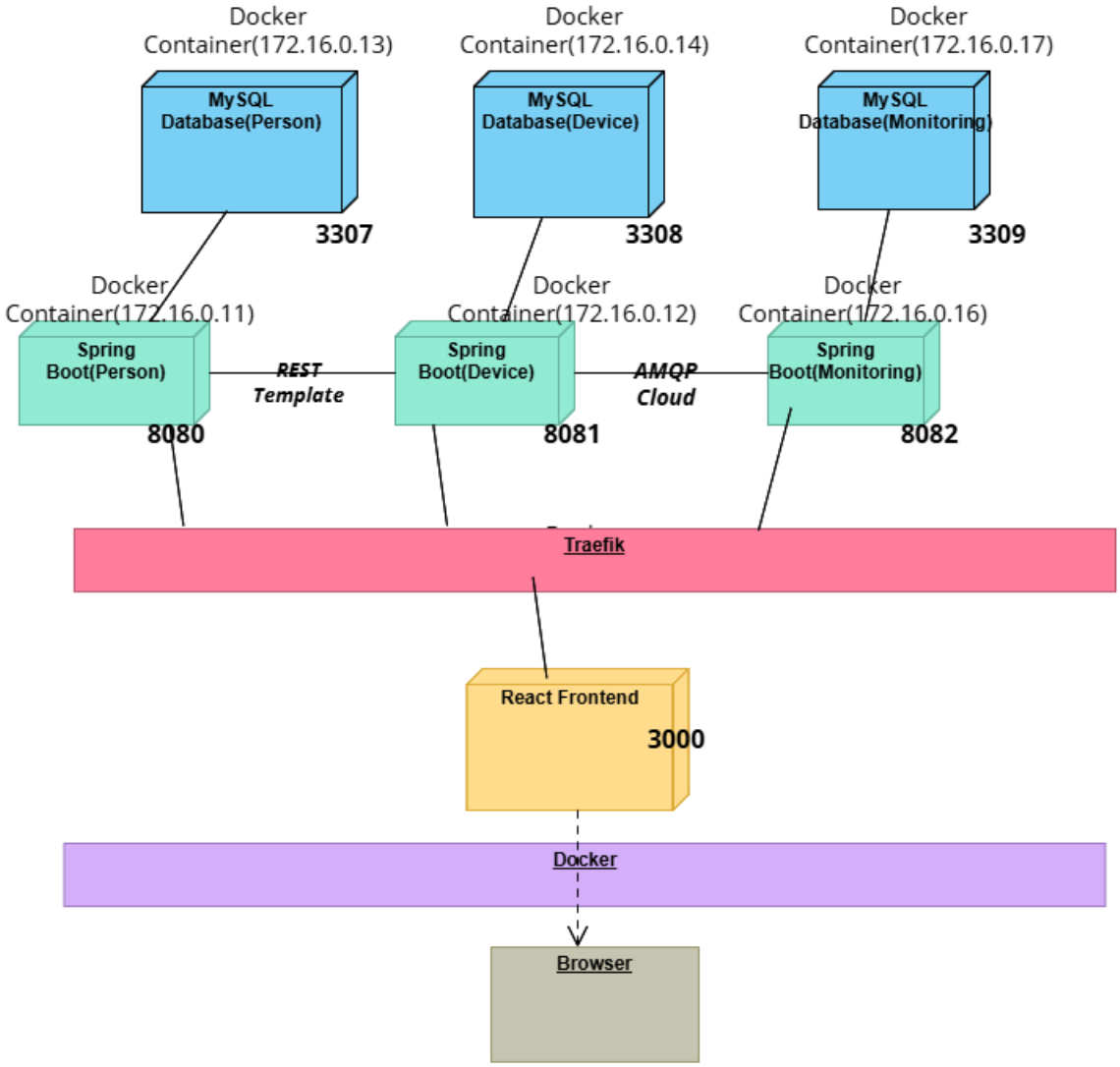

Figure 3: Deployment Diagram